

UNIVERZITA PARDUBICE · KST / KNAV · 2026/2027

Pokročilé techniky programování a návrhové vzory

Úvodní cvičení: k čemu vám předmět bude a jak to celé proběhne

PROČ TENHLE PŘEDMĚT

Návrhový vzor je osvědčený recept na problém, který se v programování pořád opakuje.

Nejde o hotový kód k opsání, ale o způsob, jak věci poskládat, aby se daly měnit.

REÁLNÉ POUŽITÍ

Se vzory se potkáte i na webu

Observer

Klik na tlačítko a stránka zareaguje.
V JavaScriptu to je
`addEventListener`.

Chain of Responsibility

Middleware: požadavek projde
přihlášením, logováním a
kontrolou práv.

Strategy

Řazení produktů v e-shopu: podle
ceny, názvu nebo hodnocení.

Builder

Dotaz na databázi skládaný po
částech: `where`, `orderBy`, `limit`.

Singleton

Jedno sdílené připojení k databázi
nebo jedna konfigurace aplikace.

Command, Memento

Zpět a vpřed (`undo` a `redo`)
v online editoru nebo
grafickém nástroji.

DOBA AI

AI napíše kód. Kdo rozhodne, jak bude poskládaný?

Vy řídíte návrh

Asistent vygeneruje funkční kód. Jak bude strukturovaný a jak se bude měnit, musíte posoudit vy.

Přesnější zadání

„Použij Strategy místo řetězce if–else“ je kratší a přesnější než deset vět popisu.

Kontrola výstupu

Poznáte, kdy AI řešení zbytečně zkomplikovalo, nebo mu chybí rozšiřitelnost.

Kdo rozumí vzorům, umí s AI pracovat jako s dobrým kolegou, ne jako s věštírnou.

PŘÍKLAD

E-shop přidává novou dopravu

Bez vzoru

```
if (method.equals("post")) price = 89;  
else if (method.equals("pickup")) price  
= 59;  
else if (method.equals("personal")) price  
= 0;  
// nová doprava = další větev
```

Každá nová doprava znamená zásah do stávajícího kódu. Takových podmínek bývá v aplikaci víc.

Se vzorem Strategy

```
Shipping shipping = new Pickup();  
double price = shipping.price(cart);  
  
// nová doprava = nová třída
```

Každá doprava je samostatná třída se společným rozhraním Shipping. Kód, který cenu počítá, se nemění.

CO SE NAUČÍTE

23 vzorů ve třech skupinách

Kreativní (5)

Mechanismy vytváření objektů, které zvyšují flexibilitu a opětovné použití kódu.

- Factory Method
- Abstract Factory
- Builder
- Singleton
- Prototype

Strukturální (7)

Jak sestavit objekty a třídy do větších struktur a zachovat je flexibilní a efektivní.

- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

Behaviorální (11)

Efektivní komunikace a přidělování odpovědnosti mezi objekty.

- Chain of Responsibility
- Command
- Interpreter
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Template Method
- Visitor

JAK TO BUDE PROBÍHAT

Čtyři cvičení, v každém jedna várka vzorů

1. cvičení

5 vzorů

Factory Method · Abstract Factory · Prototype · Builder · Singleton

2. cvičení

6 vzorů

Adapter · Bridge · Composite · Decorator · Facade · Flyweight

3. cvičení

6 vzorů

Chain of Responsibility · Command · Interpreter · Iterator · Mediator · Proxy

4. cvičení

6 vzorů + zkouška

Memento · Observer · State · Strategy · Template Method · Visitor

JEDNO CVIČENÍ

Krátce o vzorech, dlouze v kódu

1

Krátká prezentace

Lidsky, bez hloubky přednášky: k čemu je vzor dobrý a kdy po něm sáhnout.

2

Společně nad kódem

Většina času. Procházíme řešení z projektu KNAVProject: spustíme, přečteme, změníme.

3

Vaše otázky

Ptejte se, upravujte kód, zkuste si vlastní variantu.

ZKOUŠKA

O termínu rozhodnete vy

VARIANTA A

Poslední, 4. cvičení

Písemný test přímo na cvičení. Zkoušku máte za sebou už během semestru.

VARIANTA B

Řádné termíny

Vypsané termíny ve zkouškovém období, víc času na přípravu.

Vyberte si, co vám víc vyhovuje. Svou volbu si řekneme během 3. cvičení.

FORMÁT ZKOUŠKY

Písemný test, zhruba 12 otázek

~12

otázek v písemném testu

Výběr z možností

Označíte správné tvrzení.

Krátká odpověď

Vysvětlíte vlastními slovy účel nebo rozdíl vzorů.

Popis a diagram tříd

Popíšete vzor a nakreslíte jeho strukturu.

Časový limit: 60 minut · Na úspěch stačí 50 %

HODNOCENÍ

Na úspěch stačí 50 %

Známka	Procenta	Výsledek
A (1)	90 % a více	Prošli jste
B (1-)	80 až 89 %	
C (2)	70 až 79 %	
D (2-)	60 až 69 %	
E (3)	50 až 59 %	
F (4)	méně než 50 %	Neprošli jste

UKÁZKA

Takhle nějak budou otázky vypadat

VÝBĚR

Vzor Strategy umožňuje:

- zaměnit algoritmus za běhu programu
- zajistit existenci jediné instance
- zaznamenávat historii stavů objektu

KRÁTKÁ ODPOVĚĎ

Jaký je rozdíl mezi vzory Adapter a Facade?

POPIS A DIAGRAM

Co je vzor Factory Method? Popište ho a nakreslete diagram tříd.

Jde jen o ukázkou, nikoliv o skutečné otázky ze zkoušky.

PROJEKT KNAVPROJECT

Spolu se podíváme do kódu

Každý vzor má vlastní balíček s ukázkou, kterou si spustíte a přečtete.

Třída ExerciseNPoC spustí všechny ukázky daného cvičení najednou.

Čistá Java 21 bez dalších knihoven. Stačí JDK a vývojové prostředí.

```
KNAVProject/  
└─ src/exercises/  
    │─ exercise1/  
    │   │─ Exercise1PoC.java  
    │   └─ patterns/<vzor>/  
    │─ exercise2/ ...  
    │─ exercise3/ ...  
    └─ exercise4/ ...
```

Zdrojové kódy: github.com/JakubObornik/KNAVProject

PŘÍPRAVA

Co budete potřebovat



Základy OOP a Javy

Třídy, rozhraní, dědičnost a polymorfismus. Víc není potřeba.



Notebook s JDK 21 a IDE

Nejlépe IntelliJ IDEA, ale jde i jiné IDE, například NetBeans.



Stažený projekt KNAVProject

github.com/JakubObornik/KNAVProject

Otázky?

Hned po této prezentaci pokračujeme prezentací „Cvičení 1“: Factory Method, Abstract Factory, Prototype, Builder a Singleton.

Kontakt: jakub.obornik+knave@gmail.com